

PHOENIXGUARD TRADE STUDY & EXECUTION ARCHITECTURE

A source-grounded blueprint of how a broker chart becomes a studied setup, a classified opportunity, an entry decision, and — only after independent validation — an external handoff.

Version: FINAL_LIVE / V3
Prepared: 24 July 2026

EXECUTIVE POSITION

PhoenixGuard is a local-first chart intelligence and execution-control workstation. Its safe default is WAIT. A direction, confidence score, overlay, regression study, or dashboard display is not trade authority.

This document describes the repository's active V3/FINAL_LIVE architecture and its declared boundaries. It is an engineering blueprint, not financial advice, a performance claim, or an instruction to place an order.

1. Executive blueprint

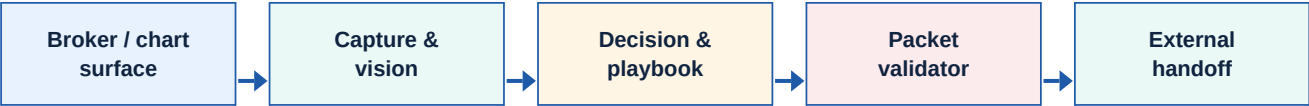
PhoenixGuard separates a high-risk workflow into independently accountable layers. It observes a locked broker/chart surface, reconstructs what is visible candle by candle, builds one explainable regression study from current and historical evidence, applies a rule-based playbook, exposes a plain-language operator story, and permits an external handoff only if a separate executable-packet contract succeeds.

CORE DOCTRINE

Observation ≠ study ≠ execution. The sole executable artefact is a fresh **PG_EXECUTION_PACKET_V3** carrying an explicit accepted **PG_ALLOWANCE_PACKAGE_V1**.

The layered design is intentional: screenshots can be stale; a visual model can be wrong; an overlay can be misaligned; historical similarity can be weak; and an apparent opportunity can be late. No single signal can jump directly to the execution edge.

The canonical authority chain



Stage	Question answered	Can it trade?
Observation	What is visibly on the locked chart?	No
Study	What conditions, paths and setups are plausible?	No
Permission	Does the current full context meet the playbook?	Not by itself
Packet validation	Is the exact permission current, internally consistent and safe to hand off?	Eligible only after pass
External boundary	Does the independent bridge accept the allowance package?	Separate downstream control

2. Runtime topology and source of truth

The canonical full-live launcher is **Backend/launch/launch_phoenixguard_live_ready.ps1**. It resolves the project-local live environment, prepares bounded runtime output, runs integrity preflight, starts the local stack, and verifies a single logical topology. The normal local API address is **127.0.0.1:8793**; the canonical session is **pocket-live-8788**.

Process / surface	Owns	Explicitly does not own
24/7 tracker orchestrator	Lifecycle, session creation, source locking and liveness	Trade permission in the launcher
FastAPI mobile API	Session state, capture services, public APIs and dashboard delivery	Browser-owned market truth
Capture worker	One locked-surface study at a time	Treating every screenshot as a candle event
Model Council + playbook	Evidence reconciliation, maturity and study/execution candidate output	Bypassing packet validation
shooter.py package reporter	Accepted allowance-package handshake	Clicking, calibration, amount edits or broker timing
MT4 / external bridge	Independent revalidation and optional file handoff	Weakening the V3 contract
Dashboard	Explainable rendering and commands	Truth, permission or episode progression

State and publication

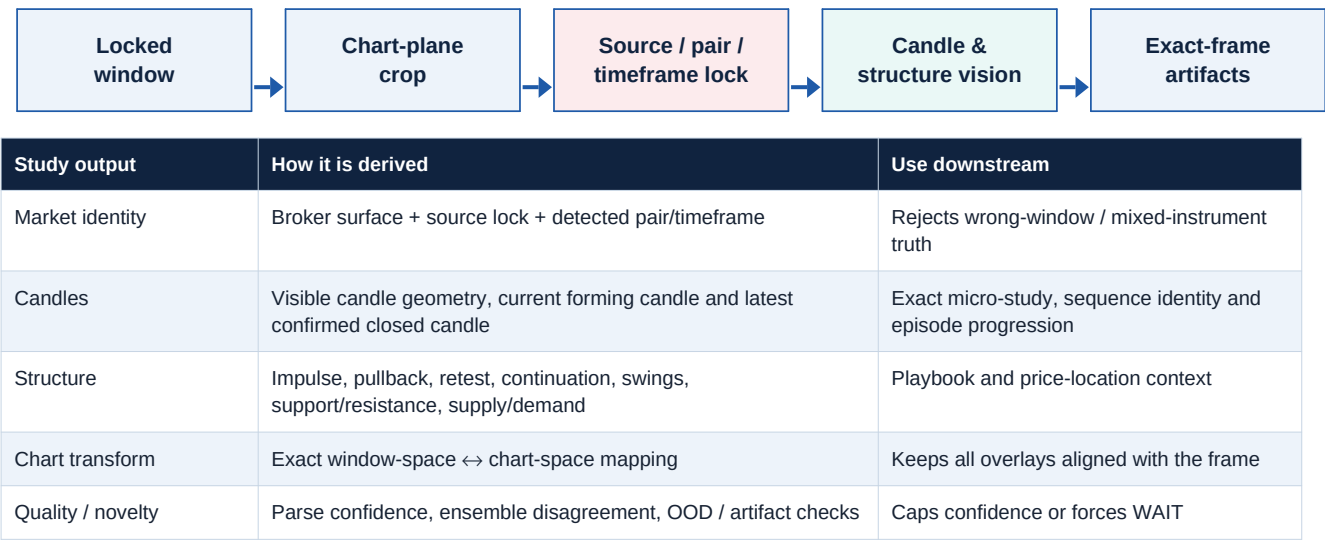
A capture is published through an atomic session commit. The exact frame's source identity, chart/window artifacts, overlay geometry, study contributors, council result, public state, and episode transition are bound together. Stale writers are rejected so a slow result cannot overwrite a newer frame. Live session artifacts are ephemeral under runtime/live; durable study memory lives under data/mobile_api/window_tracker/market_study_v3 and episode records are retained separately under data/mobile_api/window_tracker/tracking_episode_archive_v1.

FRONTEND CONTRACT

The dashboard is a renderer and command surface. It receives a privacy-safe operator workspace through SSE with bounded HTTP fallback; it does not recalculate market truth in the browser.

3. How the chart is studied: capture, identity and reconstruction

Every live study begins with a real external broker/chart window. The tracker resolves the locked handle, captures the complete window, derives or applies a chart-plane crop, records window/chart hashes, confirms market and timeframe continuity, then fails closed if the source is missing, changed, stale or ambiguous.



Closed-candle identity bridge

Tracker track_id, generic id and rolling sequence indices are positional display identifiers, not durable candle identity. For screenshot-only rows without a stable source timestamp, persistence requires exact **PG_CLOSED_CANDLE_IDENTITY_STATE_V3** proof: a stable closed-candle event key plus a non-negative monotonic event sequence for the same pair/timeframe. The resolver promotes only its confirmed current/batch rows. A prior event becomes eligible for outcome study only when it is uniquely re-observed at the exact predecessor position on the current candle axis. Shifted, replayed or arbitrarily reacquired windows cannot manufacture that proof.

Overlay and positioning discipline

Raw detections are normalized to V3 object aliases, anchored to candle geometry, clipped to chart bounds, merged and collision-checked, then rendered only against their exact frame. Real source object keys remain distinct. Pixel bounds, touch points and anchor-wick points are normalized only with the exact captured image dimensions, after which raw pixel geometry is stripped. Anonymous objects receive observation-local **OBSERVATION_ONLY** identity. Graph edges are bounded study-only observations: never inferred causation, permission or an order. Entry, target and invalidation areas must be reconstructable from the anchor frame, preserve the correct price relationship, and be hidden rather than guessed when the global re-projection fit is not proven.

- **BUY_LIMIT_ZONE**: pullback opportunity below current price; **SELL_LIMIT_ZONE**: rally opportunity above current price.
- **BUY_STOP_ENTRY_ZONE** and **SELL_STOP_ENTRY_ZONE** are prospective confirmation boundaries, not interchangeable protective stops.
- **PROTECTIVE_STOP_ZONE** records plan invalidation with explicit broker-order side and thesis side.
- **NO_VALID_ZONE** is an explicit honest classification when evidence cannot support a causal, anchored area.

4. Intelligence lanes: how a study is classified

The main image-analysis entry point is `Frontend/dashboard/main.py::run_inference`. It composes computer-vision detections, chart/sequence extraction, optional local ensemble output, memory retrieval, uncertainty models, reinforcement-learning context, 13 core skill gates plus support gates, and an ensemble decision. These models may remain internal contributors, but their old public forecast lanes are retired. Their outputs are inputs to study and council logic; they are not execution authority.

Lane	What it contributes	Authority
Computer vision	Detected patterns, candles, chart geometry, market state and reasoning trace	Observational
Scene / sequence models	Internal structure and sequence evidence from awake, versioned models	Internal contributor; retired as a public visual route
High-frequency / candle models	Near-term candle and movement context	Internal contributor; never an entry lane
Visual memory	Top-k analogous situations, similarity, ambiguity and transition tendencies	Advisory; disagreement can reduce confidence
A* scenario engine	Ranked continuation, pullback, reversal-attempt and fakeout paths	Decision-kernel evidence
Regression / RL	Quantiles, policy probabilities, online-feedback context and calibration features	Diagnostic contributor
Skill gates	Structured quality, sequence, regime and risk diagnostics	Contributor gates, not execution authority

A* scenario classification

The A* engine remains an internal challenger over CONTINUE, PULLBACK, REVERSAL_ATTEMPT and FAKEOUT transitions. It may inform the council, but Path A/Path B and double-lane public rendering are retired. The operator receives one studied BUY/SELL/HOLD regression read, with confidence and reasons, and that read still cannot become an order.

Core skill gates

The active source defines thirteen core gates: calibrated probability/conformal interval, discrete FSM state, algorithmic evidence heap, regression error estimation, knowledge representation, candle-group context, formal automata, predictive analytics and related base gates. Its router can learn relative weighting from feedback. The historical comment's consensus recipe (confidence ≥ 0.82 , at least nine gates passing and memory similarity ≥ 0.87) is a diagnostic threshold, not permission to trade.

5. Deep candle intelligence, Pair DNA and historical memory

Every proven completed candle enters a V3 study lane before any directional read is published. Price OHLC is used when available; otherwise normalized-price or pixel-price proxies remain explicitly labelled so chart pixels are never presented as broker prices or pips.

Service	What it stores or measures	Integrity boundary
Candle Intelligence V3	Exact body/range, upper and lower wicks, wick/body ratios, close location, candle type, personality, rejection, sweep, acceptance and relation to the previous candle	Closed candles only; mixed or contradictory coordinate spaces fail closed
Behavioral Sequence V3	UP swing, DOWN swing and REST states; segment candles and seconds; change, efficiency, transitions, major trend and inner trend	Current segment remains open until a later boundary proves completion
Exact candle ledger	SQLite WAL row keyed by pair, timeframe and authoritative closed-candle identity	Upsert prevents duplicate unique candles; WAL + FULL sync + immediate transaction
Pair DNA	Unique chronological candle aggregates, completed swing/rest boundaries, transitions, regimes, objects and non-causal outcome associations	Locks CLOSED_TIMESTAMP_V1 or TRACKER_EVENT_SEQUENCE_V3 per pair/timeframe; conflicting order domains are skipped
Historical similarity	Explainable 60-value sequence fingerprint, same-pair nearest sequences and supported UP/DOWN/REST outcomes	Outcome requires exact resolver N to N+1 plus unique prior-close re-observation on the current axis
Object relationship graph	Explicit candle anchors, observed-with, co-occurrence and proven normalized overlap	No inferred anchor, causal claim, order field or execution authority

The one studied direction

Major regression carries 52% of the directional study weight, inner regression 30%, and a support-qualified historical continuation 18%. Opposing evidence reduces confidence. History stays unavailable until at least three causally labelled similar cases exist, and close probabilities remain MIXED_EVIDENCE rather than being forced into BUY or SELL.

NO AUTOSCALE OR REACQUISITION LEAKAGE

A prior pixel/proxy close is never compared directly with a newly scaled frame. Positional IDs never persist. The current resolver event must be exactly prior sequence +1 and the prior close must be uniquely found again among the current frame's earlier candles; otherwise outcome maturation is skipped.

6. From classification to a trading decision

The decision kernel turns independent evidence into a coherent market read: regime, price location, path clarity, timing, risk, continuation/reversal plausibility and opposing-force room. It produces a candidate directional thesis only. The Book Strategy Master V3 then applies the complete rule set and is the final strategy decider; Model Council V3 contributes freshness, model health, source lock, sequence, timing and promotion evidence around it.



Playbook classification

Decision concept	Examples in the implementation	Effect
Playbook family	SMC turtle soup; BMS/RTO; AMD reversal; supply/demand reaction; trendline/channel; Fibonacci OTE; pivot reaction; slingshot false break; chop/no-trade	Chooses the trading grammar that must fit the observed structure
Reaction grammar	Wick rejection, body acceptance, retest hold, reclaim after sweep, continuation pressure, exhaustion, no reaction	Distinguishes a real response from a merely nearby level
Entry profile	Aggressive sniper, conservative retest, continuation retest, reversal reclaim, momentum acceptance, watch-only, no-trade	Controls the evidence expected before maturity
Maturity state	NO_OPPORTUNITY → EARLY_FORMING → VALID_WATCH → PREPARE → ENTER_NOW; or LATE_CHASE / INVALIDATED / MISSED	Makes timing explicit and prevents late relabeling

Hard blockers and fail-closed behaviors

- Stale, non-advancing or inconsistent live truth; dirty cache; missing required models; unhealthy API; invalid source lock.
- Late chase, invalidated candidate, buy-high / sell-low classification, insufficient path room or opposing force too close.
- Incomplete candle proof should remain WATCH/PREPARE rather than become a false runtime failure; it still cannot be entered.
- The public interface keeps Major trend, Inner trend, Regression study and Entry permission separate so a historical lean cannot be mistaken for a permitted trade.

7. Permission and packet validation: the real execution boundary

A permitted trade is not just BUY or SELL. The current system requires an explicit playbook ENTER_NOW decision, council evidence, a current packet, and a separate schema/runtime validation. The executable packet is built by `execution/packet_v3.py::build_execution_packet_v3` and verified by `validate_execution_packet_v3`.

Required contract group	Validation expectation	Why it exists
Identity	Session, symbol, timeframe, instrument context and source lock are present and match expected values	Prevents wrong-chart / wrong-session action
Freshness	Frame, capture and state versions advance; valid-until is in the future; fresh cache and hashes are supplied	Prevents stale action
Live integrity	is_live, frame_advancing, capture_advancing and state_advancing are true; source is model_council	Separates runtime failure from a market decision
Execution	state EXECUTABLE; side strictly BUY or SELL; expiry matches time sequence; amount action is DO_NOT_CHANGE_AMOUNT	Eliminates legacy ambiguity and local amount control
Council health	final_state/side agree; all required models awake; health provenance is carried	Avoids unsupported/misaligned permission
Allowance	Explicit PG_ALLOWANCE_PACKAGE_V1, accepted and execution-ready	Makes handoff scope explicit

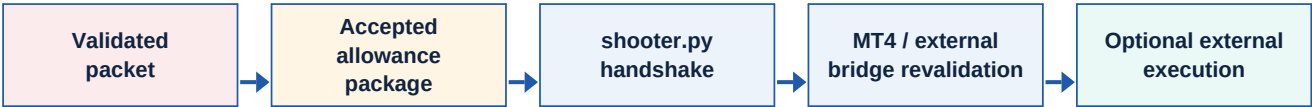
WHAT IS REJECTED

Raw action / execution_action payloads, old schemas, STUDY_PACKETs, CALL/PUT aliases, missing final side, stale packets, non-advancing frames and identity mismatches are not executable packets.

The packet’s authority metadata carries `PLAYBOOK_FINAL_DECIDER_V3` as strategy authority and `PG_EXECUTION_PACKET_V3` as packet authority. Packet validation classifies freshness, identity, cache, live-state and model-awake failures as `RUNTIME_INTEGRITY`—not as a reason to reinterpret unsafe state as a market blocker.

8. What ‘traded’ means in FINAL_LIVE

In the canonical FINAL_LIVE setup, direct local broker clicking is disabled. The launcher may enable live evaluation and construction of permission state, while PHOENIXGUARD_ALLOW_LIVE_BROKER_CLICKS remains disabled. The historical shooter is now a package reporter, not a clicker.



Component	Behavior at the handoff edge
Execution packet endpoint	Exposes executable packets only; a visible study is deliberately not enough.
shooter.py	Fetches the V3 packet, rejects absent/stale/malformed/contradictory payloads, and writes a bounded accepted-package handshake only.
Allowance package	Must be explicit, accepted, execution-ready and of a supported type such as INTRADAY_ENTER_NOW or SWING.
MT4 file bridge	Compacts and revalidates the allowance. It rejects inferred, missing, non-accepted, non-ready or non-professional packages before any command handoff.
Broker amount / time	The execution contract carries DO_NOT_CHANGE_AMOUNT; the local reporter does not calibrate controls, edit amount or set broker timing.

IMPORTANT BOUNDARY

The repository proves a controlled handoff architecture, not a promise of broker fill, price, profit or loss avoidance. Broker rules, spread, slippage, trigger conventions and fill mechanics remain external and variable.

9. Tracking episodes: candle-by-candle regression history

PhoenixGuard records a fixed 12-newly-completed-candle episode, not repeated screen refreshes. At Start, a unique episode ID freezes the anchor: market/timeframe, closed candle, committed plan/thesis, study baseline, permission snapshot, candidate positioning areas and a 12-event horizon. Each event then records actual movement, rest, continuation or change without rewriting the earlier study.

State	Meaning	Transition
IDLE	No active episode	Start after readiness
ACTIVE	Frozen baseline is being compared to new confirmed closed candles	E1–E11 each append exactly once
COMPLETED	E12 recorded; before/after record retained	New operator start creates next episode
STOPPED	Only the episode stops; worker and models stay warm	Restart after operator action
INVALIDATED / FAILED	Market/timeframe/source changed or unrecoverable fault	Restart only after readiness is restored

One event only advances when the episode is ACTIVE, pair/timeframe match, a confirmed closed-candle key exists, that key has not been processed, and its sequence is strictly newer. A historical event receives a market-study snapshot only when both its closed-candle key and sequence match exactly. A current study is never copied backward onto older reacquired events; a missing study remains honestly unavailable.

Continuity handling

Missed visual rollovers enter a fail-closed reacquisition path. Unknown gaps can remain recorded as gaps, while stable new candle identities resume progression. This protects history from double counting and from falsely attaching the latest regression to an earlier candle.

10. Operator experience, auditability and operational controls

The privacy-safe public object is PG_OPERATOR_WORKSPACE_V1. It exposes only what an operator needs: market/timeframe, major trend, inner trend, historical regression, independent entry permission with a blocking reason, freshness, E1–E12 state/history, exact-frame media and safe overlay metadata. It removes provider internals, filesystem details, raw telemetry and private strategy vocabulary.

Operator-facing idea	Internal implementation principle
Major trend	Larger completed-candle regression and structure
Inner trend	Current swing, pullback, rest or continuation inside the major trend
Regression study	One supported BUY/SELL/HOLD historical read with reasons; never permission
Entry status	Separate permission contract that fails closed if packet, freshness, integrity or entry-area conditions fail
Buy low / sell high guidance	Positioning guidance inside a valid area; never overrides invalidation, freshness or entry-window gates
History	Ghosted retained episode evidence; never changes current permission

Audit and certification

- Runtime artifacts: session.json, compact_live_state.json, display_state.json, events.jsonl, tracking episode state/events, exact frame media and overlays.
- Evidence loop: allowed-entry and blocked-ENTER_NOW screenshots, future outcome scoring, progression galleries, manifests and forensic reports.
- Health and topology: API health, single runtime/venv validation, model warm state, source lock, dashboard hydration, broker freshness and runtime traces.
- Useful live endpoints: /v1/mobile/health; operator/state/v1/{session}; model-council execution/latest; runtime/trace/v3; and the /v3 dashboard.

OPERATING RULE

Treat an absent execution packet as a safe waiting state unless diagnostics say otherwise. A study packet can be valuable evidence while correctly remaining non-executable.

11. Implementation map and verification checklist

The following files are the most relevant active surfaces for understanding or changing the system. This blueprint intentionally prioritizes current source and canonical blueprints over archived architecture snapshots.

Area	Primary active source / canonical reference
Launch / topology	Backend/launch/launch_phoenixguard_live_ready.ps1; start_phoenixguard_full_local.ps1; start_phoenixguard_24_7_tracker.py
Live tracker	Backend/src/phoenixguard/mobile_api/window_tracker.py
Closed-candle resolver	phoenixguard/decision/scene_forecast_contributor_v3.py; mobile_api/window_tracker.py resolver-to-study bridge
Deep V3 study	phoenixguard/study/candle_intelligence_v3.py; behavioral_sequence_v3.py; candle_ledger_v3.py; pair_dna_v3.py; historical_similarity_v3.py; object_relationship_graph_v3.py
API / public state	Backend/src/phoenixguard/mobile_api/app.py; live_state_v3.py
Inference	Frontend/dashboard/main.py::run_inference
Vision / overlays	phoenixguard/vision/*; tracking/market_object_tracker_v3.py; vision/v3_overlay_contract.py
Decision	phoenixguard/decision/model_council_v3.py; book_strategy/*; scenario_decision_kernel.py; a_star_scenarios.py; skill_gates.py
Execution boundary	phoenixguard/execution/packet_v3.py; v3_language.py; Backend/launch/shooter.py; phoenixguard_mt4_file_bridge.py
Dashboard	Frontend/dashboard/static/window_tracker_dashboard.html
Canonical specs	docs/architecture/PHOENIXGUARD_LIVE_TRACKING_BLUEPRINT.md; PhoenixGuard_System_Blueprint.md; docs/active_execution_paths.md; docs/execution_packet_schema_matrix.md

Before considering a live stack healthy

- Health returns 200 and exactly one logical stack owns the configured port.
- The session is running; frame/capture/state identities advance across samples; pair and timeframe remain stable.
- Latest window/chart/overlay artifacts, operator state and dashboard all return successfully and agree on version identity.
- Tracking readiness is explicit; a started episode freezes a baseline; only genuinely new closed candles advance it.
- Model health, source lock, packet freshness and package handoff status are checked separately from market analysis.
- No regression read, overlay or raw signal is treated as entry permission; only a fresh validated V3 packet plus accepted allowance may cross the handoff boundary.

BOTTOM LINE

PhoenixGuard is architected as an evidence-to-permission system, not a model-to-click system. The safest reading of the product is: study broadly, classify explicitly, wait by default, and permit a handoff only through independently validated current truth.